

**INSTITUTO TECNOLÓGICO DE BUENOS AIRES
ESCUELA DE GESTIÓN Y TECNOLOGÍA**

Pokemonad

Motor de Juego de Estrategia Multijugador

AUTORES: Francisco Quian Blanco
Theo Stanfield

PROFESORES: Pablo Ernesto Martinez Lopez

TRABAJO FINAL:
72.60 - Programación Funcional - 20252Q

CIUDAD AUTÓNOMA DE BUENOS AIRES
Mayo 2026

Índice

1. Introducción	2
1.1. Contexto y Motivación	2
1.2. Objetivos del Proyecto	2
1.3. Integrantes y Metodología de Trabajo	3
2. Marco Teórico y Conceptos de Programación Funcional	4
2.1. Características del Paradigma a poner en juego	4
2.2. Desafíos en Programación Funcional y Uso de Mónadas	5
2.3. Herramientas y Bibliotecas Utilizadas	6
3. Arquitectura del Motor (Solución Propuesta)	8
3.1. Visión General del Sistema	8
3.2. Gestión del Estado Inmutable	8
3.3. Ciclo de Vida del Juego (Game Loop)	8
4. Detalles de Diseño e Interacción con el Medio	8
4.1. Modelado de Datos	8
4.2. El Rol de las Mónadas en la Interacción	8
4.3. Lógica Multijugador y Concurrency	8
5. Conclusiones	8
5.1. Evaluación del Alcance y Resultados	8
5.2. Dificultades Encontradas y Resoluciones	8
5.3. Lecciones Aprendidas	8
5.4. Trabajo Futuro	8

1. Introducción

1.1. Contexto y Motivación

El presente trabajo describe el desarrollo de **Pokemonad**, un motor de juego de combate por turnos inspirado en la mecánica clásica de Pokémon, construido íntegramente utilizando el lenguaje de programación funcional Haskell.

La motivación principal del proyecto no radica en la réplica exhaustiva de las complejas reglas del juego original, sino en utilizar este dominio interactivo como vehículo para explorar y resolver problemas de ingeniería de software dentro del paradigma funcional. Tras la etapa de ideación y evaluación de alcance, el enfoque del proyecto se centró en dos grandes desafíos técnicos: la implementación de una arquitectura de red descentralizada Peer-to-Peer (P2P) concurrente y segura, y el desarrollo de una Inteligencia Artificial no trivial para el modo de un solo jugador.

Para lograr la profundidad requerida en estos subsistemas, se tomó la decisión arquitectónica de mantener la lógica de dominio del combate en su forma más pura y básica. Las interacciones se restringieron exclusivamente a acciones de ataque directo (*FIGHT*) y cambio de criaturas (*SWITCH*), omitiendo alteraciones de estado, modificadores de estadísticas y habilidades pasivas. Esta reducción deliberada del alcance permite concentrar el esfuerzo de desarrollo en la gestión del estado concurrente y en los árboles de decisión del bot oponente.

1.2. Objetivos del Proyecto

El objetivo general del trabajo es demostrar la aplicación práctica de conceptos avanzados de programación funcional, tales como la inmutabilidad, la evaluación de funciones puras, y el manejo de efectos a través de mónadas, en la construcción de un sistema de software interactivo y distribuido.

Los objetivos específicos, acordados y delimitados para el alcance de este trabajo, incluyen:

- **Desarrollo de una biblioteca P2P independiente:** Diseñar un módulo de comunicación de bajo nivel mediante sockets TCP puro. Este subsistema gestiona la recepción asincrónica de flujos de bytes (con serialización binaria en tramas) y delega la sincronización de estados concurrentes mediante Memoria Transaccional en Software (STM, usando `TQueue`). Este módulo opera de forma completamente desacoplada de las reglas de *Pokemonad*, constituyendo una biblioteca genérica.

- **Implementación de un oponente controlado por IA:** Construir un agente inteligente para el modo de un jugador que supere la toma de decisiones heurística trivial. Se implementó un modelo basado en **Aprendizaje por Refuerzo (Reinforcement Learning)**, utilizando una aproximación lineal para estimar los valores Q (Q-Values) a partir de la extracción de características del entorno (ventaja de tipos, proyecciones de daño y viabilidad de cambio de criaturas). La selección de acciones se rige por una política ϵ -greedy parametrizada según la dificultad del oponente.
- **Integración de una Interfaz Gráfica (GUI):** Proveer una capa visual interactiva utilizando la biblioteca `gloss`. Esta herramienta permite renderizar gráficos 2D de forma declarativa, exponiendo el estado inmutable del motor en tiempo real e integrándose fluidamente con el bucle principal de la aplicación.

1.3. Integrantes y Metodología de Trabajo

El proyecto fue diseñado y desarrollado íntegramente por Francisco Quian Blanco y Theo Stanfield.

Dada la complejidad inherente de orquestar un hilo de red asincrónico junto con un hilo de interfaz gráfica (UI) evitando condiciones de carrera, se adoptó una metodología de desarrollo colaborativa y altamente comunicativa. La división de tareas se gestionó de manera dinámica apoyándose en un sistema de control de versiones.

Para los componentes críticos de la arquitectura —particularmente el aislamiento de la biblioteca P2P y la implementación de la IA— se priorizó el trabajo conjunto mediante sesiones de *pair programming*. Para el desarrollo de módulos individuales, se establecieron revisiones de código cruzadas (*code reviews*). Esta metodología garantiza no solo el avance equilibrado del proyecto, sino que asegura que ambos integrantes mantengan un conocimiento total y un manejo profundo de cada aspecto del código fuente producido, cumpliendo rigurosamente con los requisitos de evaluación.

2. Marco Teórico y Conceptos de Programación Funcional

El desarrollo de un motor interactivo como **Pokemonad** en Haskell exige aplicar los pilares del paradigma funcional puro para garantizar un código predecible, testeable y seguro [3].

2.1. Características del Paradigma a poner en juego

Las características centrales puestas en juego son:

- **Inmutabilidad:** A diferencia de los lenguajes imperativos donde el estado de un objeto se sobrescribe en memoria, en Haskell las estructuras de datos son inmutables. En el proyecto, cuando un Pokémon recibe daño, no se modifica su variable interna de puntos de vida. En su lugar, el módulo `Pokemonad.Battle.State` genera una copia completamente nueva del estado de la batalla que refleja la reducción de salud, preservando el estado anterior de forma intacta.
- **Funciones Puras:** Una función pura garantiza que, para los mismos argumentos de entrada, siempre retornará el mismo resultado sin producir efectos secundarios. Los cálculos delegados a `Pokemonad.Battle.Logic` y `Pokemonad.Battle.Damage` son puros; el daño resultante de un ataque depende estrictamente de las estadísticas del atacante, del defensor y de las propiedades del movimiento, lo que permite razonar matemáticamente sobre el combate y facilita enormemente las pruebas unitarias.
- **Tipos de Datos Algebraicos (ADTs):** Los ADTs permiten modelar el dominio del problema de forma expresiva y segura en tiempo de compilación. A través de módulos como `Pokemonad.Core.Types`, `Pokemonad.Core.Move` y `Pokemonad.Core.Pokemon`, el sistema representa exhaustivamente los tipos elementales, los movimientos y los estados de los entrenadores (`Pokemonad.Core.Trainer`), evitando estados inválidos o transiciones ilógicas.
- **Funciones de Orden Superior y Composición:** La capacidad de pasar funciones como parámetros o retornarlas permite construir lógicas complejas a partir de piezas simples. En la evaluación de la Inteligencia Artificial (`Pokemonad.AI.Model` y `Pokemonad.AI.Decision`), las funciones de orden superior extraen y ponderan características del entorno de forma declarativa para construir árboles de decisión limpios [2].

- **Evaluación Perezosa (Lazy Evaluation):** Haskell no evalúa una expresión hasta que su resultado es estrictamente necesario. En el contexto de la simulación de turnos o en la generación de ramificaciones de decisiones para la IA, la evaluación perezosa permite definir espacios de estados potencialmente gigantescos sin incurrir en costos de memoria o procesamiento inmediatos, computando únicamente el camino que el motor decide explorar.

2.2. Desafíos en Programación Funcional y Uso de Mónadas

Construir un videojuego interactivo y multijugador bajo el rigor funcional impone desafíos de diseño, fundamentalmente al lidiar con mutabilidad, interacción externa y concurrencia. Para abordar esto, `Pokemonad` implementa un patrón de diseño conocido como “Pure Core + Thin IO Layer”, separando estrictamente el `game-engine` (núcleo puro) del `game-client` y `p2p-net` (capas de interacción).

Las herramientas fundamentales para resolver estos desafíos son las **mónadas**, las cuales encapsulan el contexto computacional de las operaciones:

- **Modelado del Estado Dinámico (Mónadas State y Transformadores StateT):** Simular el cambio de estado turno a turno requiere propagar estructuras complejas. La biblioteca `mtl` [1] permite utilizar `State` o `StateT` para enhebrar implícitamente el contexto del juego (`Pokemonad.Battle.State` y `Client.State`) a través de las operaciones. Esto otorga la conveniencia sintáctica de la mutabilidad secuencial sin sacrificar la pureza matemática subyacente.
- **Interacción con el Medio (Mónada IO):** Todo efecto secundario (dibujar en pantalla, leer el teclado o acceder al disco) debe confinarse en `IO`. La capa visual manejada en `Client.Render` y `Client.Drawing`, así como la lectura de archivos de entrenamiento como `data/ai_checkpoint.txt`, ocurren dentro de este contexto asilado, protegiendo al motor puro de inconsistencias.
- **Concurrencia y Sincronización Multijugador (Mónada STM):** El mayor desafío del multijugador P2P es gestionar la memoria compartida entre el hilo de red (escuchando sockets) y el hilo gráfico (renderizando el juego) sin incurrir en condiciones de carrera. El proyecto utiliza `STM` (Software Transactional Memory) [4] para garantizar que los intercambios de estado y eventos en la red ocurran de manera atómica, coordinada y sin bloqueos explícitos (deadlocks).

- **Aleatoriedad en un Entorno Puro:** La generación de eventos probabilísticos requiere romper el determinismo trivial. Esto se resuelve pasando explícitamente generadores pseudoaleatorios (`random`) a través de las funciones puras en `Pokemonad.Battle.Turn` y los módulos de decisión de la IA, garantizando que la simulación completa sea reproducible.
- **Manejo de Errores y Fallos (Mónadas `Either` y `Maybe`):** Para la comunicación en red a través de `P2P.Serialization` y `Client.NetSerializers`, el parseo de bytes entrantes puede fallar. En lugar de lanzar excepciones que rompan el flujo de ejecución, se utilizan contextos como `Either` o `Maybe` para representar de forma explícita y segura el éxito o fracaso de la decodificación.

2.3. Herramientas y Bibliotecas Utilizadas

Para materializar esta arquitectura, el proyecto se apoya en un ecosistema robusto gestionado mediante **Cabal** [5], la herramienta estándar de Haskell para definir dependencias, compilar y organizar la solución en tres paquetes interconectados (`game-engine`, `p2p-net` y `game-client`).

- **network:** Provee el acceso de bajo nivel a los sockets TCP en `P2P.Communication`, estableciendo los canales directos requeridos para la arquitectura Peer-to-Peer.
- **stm:** Fundamental para la arquitectura asincrónica. Proporciona las primitivas de memoria transaccional para coordinar de forma segura el estado del cliente y los mensajes recibidos a través de la red.
- **binary y bytestring:** Utilizados en `P2P.Serialization` y `Client.NetSerializers`. Permiten manipular secuencias de bytes crudas y proveen las clases de tipos necesarias para empaquetar y desempaquetar eficientemente los mensajes del protocolo para su envío por la red.
- **mtl (Monad Transformer Library):** Extensión esencial empleada exhaustivamente en el `game-engine` para combinar mónadas sin acoplar lógicas de infraestructura a la lógica de negocio.
- **random:** Implementa la lógica de generación de números pseudoaleatorios, indispensable tanto para la incertidumbre del combate como para las dinámicas de exploración en el entrenamiento de la Inteligencia Artificial (`Pokemonad.AI.Training`).

- **gloss y gloss-juicy**: Ecosistema gráfico funcional elegido para **game-client**. **gloss** permite renderizar escenas 2D de forma puramente declarativa conectándolas a modelos de estado, mientras que **gloss-juicy** extiende estas capacidades permitiendo cargar y manipular imágenes rasterizadas complejas alojadas en la carpeta **assets**.
- **containers y text**: Proporcionan estructuras de datos eficientes (como mapas) utilizadas para indexar elementos, y permiten manejar cadenas de caracteres Unicode con alta eficiencia de memoria en la interfaz, superando a las listas de caracteres nativas.

La elección de programación funcional pura, soportada por mónadas transaccionales y de estado, garantiza que la complejidad inherente a un sistema multijugador concurrente y de comportamiento no determinista permanezca manejable. Estas bases teóricas sientan el terreno propicio para comprender la división arquitectónica y las decisiones de diseño estructural de las siguientes secciones.

3. Arquitectura del Motor (Solución Propuesta)

3.1. Visión General del Sistema

3.2. Gestión del Estado Inmutable

3.3. Ciclo de Vida del Juego (Game Loop)

4. Detalles de Diseño e Interacción con el Medio

4.1. Modelado de Datos

4.2. El Rol de las Mónadas en la Interacción

4.3. Lógica Multijugador y Concurrencia

5. Conclusiones

5.1. Evaluación del Alcance y Resultados

5.2. Dificultades Encontradas y Resoluciones

5.3. Lecciones Aprendidas

5.4. Trabajo Futuro

Referencias

- [1] Haskell.org. *Monad Transformer Library (mtl) Documentation*, 2026. Consultado en línea.
- [2] Graham Hutton. *Programming in Haskell*. Cambridge University Press, 2nd edition, 2016.
- [3] Simon Marlow et al. *The Haskell 2010 Language Report*. Haskell.org, 2010.
- [4] Simon Peyton Jones. Beautiful concurrency. In Andy Oram and Greg Wilson, editors, *Beautiful Code*. O'Reilly Media, 2007.

- [5] Cabal Development Team. *The Cabal User Guide*, 2026. Consultado en línea.